

part_a_classical

January 20, 2026

1 Classical Machine Learning for Video Actions

This script implements and compares three classical machine learning algorithms using extracted video features:

1. Support Vector Machine (Linear + RBF)
2. Random Forest Classifier
3. k-Nearest Neighbors (k-NN)

Features are extracted using existing project modules: - data_loader.py - feature_extraction.py

Author: Student_2024AB05275

```
[1]: pip install -r requirements.txt
```

```
Requirement already satisfied: numpy>=1.23 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 23)) (1.24.4)
```

```
Requirement already satisfied: scipy>=1.10 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 31)) (1.11.3)
```

```
Requirement already satisfied: torch>=2.0 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 42)) (2.3.0+cu121)
```

```
Requirement already satisfied: torchvision>=0.15 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 49)) (0.18.0+cu121)
```

```
Requirement already satisfied: openpyxl>=3.1.0 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 54)) (3.1.5)
```

```
Requirement already satisfied: opencv-python-headless in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 69)) (4.11.0.86)
```

```
Requirement already satisfied: scikit-learn>=1.3 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 77)) (1.8.0)
```

```
Requirement already satisfied: scikit-image in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 84)) (0.22.0)
```

```
Requirement already satisfied: matplotlib>=3.7 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 96)) (3.8.4)
```

```
Requirement already satisfied: seaborn>=0.13 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 103)) (0.13.2)
```

Requirement already satisfied: umap-learn in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 110)) (0.5.11)

Requirement already satisfied: tqdm>=4.66 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 122)) (4.66.4)

Requirement already satisfied: filelock in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.13.1)

Requirement already satisfied: typing-extensions>=4.8.0 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (4.12.2)

Requirement already satisfied: sympy in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (1.12)

Requirement already satisfied: networkx in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.3)

Requirement already satisfied: jinja2 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.1.4)

Requirement already satisfied: fsspec in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2024.2.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (8.9.2.26)

Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.3.1)

Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (11.0.2.54)

Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (10.3.2.106)

Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (11.4.5.107)

Requirement already satisfied: nvidia-cuspars-cu12==12.1.0.106 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.0.106)

Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2.20.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in

/opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt
 (line 42)) (12.1.105)
 Requirement already satisfied: triton==2.3.0 in /opt/conda/lib/python3.11/site-
 packages (from torch>=2.0->-r requirements.txt (line 42)) (2.3.0)
 Requirement already satisfied: nvidia-nvjitlink-cu12 in
 /opt/conda/lib/python3.11/site-packages (from nvidia-cusolver-
 cu12==11.4.5.107->torch>=2.0->-r requirements.txt (line 42)) (12.1.105)
 Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
 /opt/conda/lib/python3.11/site-packages (from torchvision>=0.15->-r
 requirements.txt (line 49)) (10.4.0)
 Requirement already satisfied: et-xmlfile in /opt/conda/lib/python3.11/site-
 packages (from openpyxl>=3.1.0->-r requirements.txt (line 54)) (2.0.0)
 Requirement already satisfied: joblib>=1.3.0 in /opt/conda/lib/python3.11/site-
 packages (from scikit-learn>=1.3->-r requirements.txt (line 77)) (1.4.2)
 Requirement already satisfied: threadpoolctl>=3.2.0 in
 /opt/conda/lib/python3.11/site-packages (from scikit-learn>=1.3->-r
 requirements.txt (line 77)) (3.5.0)
 Requirement already satisfied: imageio>=2.27 in /opt/conda/lib/python3.11/site-
 packages (from scikit-image->-r requirements.txt (line 84)) (2.34.2)
 Requirement already satisfied: tifffile>=2022.8.12 in
 /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt
 (line 84)) (2024.7.2)
 Requirement already satisfied: packaging>=21 in /opt/conda/lib/python3.11/site-
 packages (from scikit-image->-r requirements.txt (line 84)) (24.1)
 Requirement already satisfied: lazy_loader>=0.3 in
 /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt
 (line 84)) (0.4)
 Requirement already satisfied: contourpy>=1.0.1 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (1.2.1)
 Requirement already satisfied: cycycler>=0.10 in /opt/conda/lib/python3.11/site-
 packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (4.53.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (1.4.5)
 Requirement already satisfied: pyparsing>=2.3.1 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (3.1.2)
 Requirement already satisfied: python-dateutil>=2.7 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (2.9.0)
 Requirement already satisfied: pandas>=1.2 in /opt/conda/lib/python3.11/site-
 packages (from seaborn>=0.13->-r requirements.txt (line 103)) (2.1.4)
 Requirement already satisfied: numba>=0.51.2 in /opt/conda/lib/python3.11/site-
 packages (from umap-learn->-r requirements.txt (line 110)) (0.63.1)

Requirement already satisfied: pynndescent>=0.5 in /opt/conda/lib/python3.11/site-packages (from umap-learn->-r requirements.txt (line 110)) (0.6.0)

Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in /opt/conda/lib/python3.11/site-packages (from numba>=0.51.2->umap-learn->-r requirements.txt (line 110)) (0.46.0)

Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.11/site-packages (from pandas>=1.2->seaborn>=0.13->-r requirements.txt (line 103)) (2024.1)

Requirement already satisfied: tzdata>=2022.1 in /opt/conda/lib/python3.11/site-packages (from pandas>=1.2->seaborn>=0.13->-r requirements.txt (line 103)) (2024.1)

Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib>=3.7->-r requirements.txt (line 96)) (1.16.0)

Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.11/site-packages (from jinja2->torch>=2.0->-r requirements.txt (line 42)) (2.1.5)

Requirement already satisfied: mpmath>=0.19 in /opt/conda/lib/python3.11/site-packages (from sympy->torch>=2.0->-r requirements.txt (line 42)) (1.3.0)

Note: you may need to restart the kernel to use updated packages.

```
[11]: # =====
# Standard Library Imports
# =====
import os
import time
from typing import Dict, Tuple
import warnings

# =====
# Third-Party Imports
# =====
import numpy as np
import pandas as pd
from pathlib import Path
# Object-oriented, cross-platform file system path handling
import matplotlib.pyplot as plt

from evaluate import evaluate_classification

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```

        recall_score,
        f1_score,
        confusion_matrix,
        ConfusionMatrixDisplay,
    )

    from sklearn.svm import SVC
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.neighbors import KNeighborsClassifier
    from IPython.display import display, Markdown

    # =====
    # Project Imports
    # =====
    from feature_extraction import (
        visualize_classical_feature_importance,
        visualize_classical_feature_space,
        get_classical_feature_names
    )
    from data_loader import load_split_data_and_extract_features
    from evaluate import create_full_comparison_table, generate_dynamic_observations

```

```

[3]: # =====
      # Global Configurations
      # =====
      warnings.filterwarnings("ignore")
      RANDOM_STATE = 42
      CV_FOLDS = 5

      # =====
      # Utility Functions
      # =====
      def evaluate_model(
          model,
          X_test: np.ndarray,
          y_test: np.ndarray,
      ) -> Dict[str, float]:
          """
          Evaluate a trained model using multiple metrics.

          Returns:
              Dictionary containing accuracy, precision, recall, and F1-score.
          """
          y_pred = model.predict(X_test)

          return {

```

```

        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, average="macro"),
        "recall": recall_score(y_test, y_pred, average="macro"),
        "f1_score": f1_score(y_test, y_pred, average="macro"),
    }

def plot_confusion_matrix(model, X_test, y_test, title: str) -> None:
    """
    Plot confusion matrix for a trained classifier.
    """
    cm = confusion_matrix(y_test, model.predict(X_test))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot()
    plt.title(title)
    plt.show()

```

```

[4]: # =====
# Data Loading & Feature Extraction
# =====
print("\n Loading dataset splits and extract features...")

print("\n-----")
print("* Processing training dataset.....")
print("-----\n")
X_train, y_train, CLASS_TO_LABEL, LABEL_TO_CLASS = \
    load_split_data_and_extract_features("train")

print("\n-----")
print("* Processing validation dataset.....")
print("-----\n")
X_val, y_val, _, _ = load_split_data_and_extract_features(split_name="val")

print("\n-----")
print("* Processing test dataset.....")
print("-----\n")
X_test, y_test, _, _ = load_split_data_and_extract_features(split_name="test")

print(f"\n\n* Train shape: {X_train.shape}")
print(f"* Test shape : {X_test.shape}")

```

Loading dataset splits and extract features...

 * Processing training dataset...

Detected classes (5): ['class_1_Diving', 'class_2_JumpRope',
'class_3_SkyDiving', 'class_4_Surfing', 'class_5_Swing']

```
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g10_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g19_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g14_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g17_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g02_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g18_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g22_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g02_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g19_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g04_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g02_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g01_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g24_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g23_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g17_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g07_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g08_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g07_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g10_c06.avi
```


[illegible]

[illegible]

[illegible]

Detected classes (5): ['class_1_Diving', 'class_2_JumpRope',
'class_3_SkyDiving', 'class_4_Surfing', 'class_5_Swing']

```
[INFO] * processing Video file : class_1_Diving/v_Diving_g07_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g22_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g14_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g01_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g11_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g01_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g11_c06.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g14_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g18_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g11_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g23_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g05_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g15_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g11_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g16_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g06_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g07_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g18_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g21_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g01_c07.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g25_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g23_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g01_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g13_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g07_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g16_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g13_c02.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g20_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g18_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g03_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g17_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g05_c07.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g21_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g25_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g19_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g02_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g08_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g10_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g17_c03.avi
```

```
[INFO] * processing Video file : class_5_Swing/v_Swing_g05_c05.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g23_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g25_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g15_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g09_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g02_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g09_c07.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g01_c02.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g19_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g09_c02.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g18_c07.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g22_c02.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g10_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g05_c01.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g18_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g08_c01.avi
```

* Processing test dataset...

Detected classes (5): ['class_1_Diving', 'class_2_JumpRope',
'class_3_SkyDiving', 'class_4_Surfing', 'class_5_Swing']

```
[INFO] * processing Video file : class_1_Diving/v_Diving_g04_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g14_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g24_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g17_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g08_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g19_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g22_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g25_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g05_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g14_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g12_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g11_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g06_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g09_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g13_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g09_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g01_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g03_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g25_c01.avi
```

```

[INFO] * processing Video file : class_4_Surfing/v_Surfing_g24_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g13_c05.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g14_c02.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g17_c05.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g17_c07.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g19_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g24_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g04_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g08_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g09_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g10_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g11_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g23_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g24_c05.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g06_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g16_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g12_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g14_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g07_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g24_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g04_c07.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g13_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g07_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g25_c01.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g21_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g19_c01.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g12_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g08_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g04_c07.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g12_c05.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g07_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g21_c05.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g05_c05.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g22_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g16_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g20_c02.avi

```

* Train shape: (280, 399)

* Test shape : (60, 399)

```

[5]: # =====
# 1 SUPPORT VECTOR MACHINE (LINEAR + RBF)
# =====
# This block trains an SVM classifier using GridSearchCV
# and performs post-training feature analysis including:
# - Feature importance (linear kernel only)

```

```

# - t-SNE visualization
# - UMAP visualization
# =====

print("\n-----")
print("  Training Support Vector Machine...")
print("-----\n")

# -----
#   Step 1: Define SVM pipeline
# -----
# StandardScaler is REQUIRED for SVM to:
# - normalize feature magnitudes
# - ensure stable optimization
# - improve convergence
# -----

svm_pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        (
            "svm",
            SVC(
                probability=True,    # Required for ROC/AUC computation
                random_state=42
            ),
        ),
    ]
)

# -----
#   Step 2: Define hyperparameter search space
# -----
# - Linear kernel → supports feature importance via coefficients
# - RBF kernel → non-linear, higher capacity but no interpretability
# -----

svm_param_grid = [
    {
        "svm__kernel": ["linear"],
        "svm__C": [0.1, 1, 10],
    },
    {
        "svm__kernel": ["rbf"],
        "svm__C": [0.1, 1, 10],
        "svm__gamma": [0.01, 0.1, 1],
    },
]

```



```

]

# -----
#   Step 3: GridSearchCV setup
# -----
# - Cross-validation ensures robust hyperparameter selection
# - Accuracy is used as optimization metric
# -----

svm_grid = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=svm_param_grid,
    cv=CV_FOLDS,
    scoring="accuracy",
    n_jobs=-1,
    verbose=1,
)

# -----
#   Step 4: Measure training time
# -----

train_start_time = time.perf_counter()
svm_grid.fit(X_train, y_train)
train_end_time = time.perf_counter()

svm_training_time = train_end_time - train_start_time

print(f"\n* SVM Training Time: {svm_training_time:.4f} seconds")
print("* Best SVM Parameters:", svm_grid.best_params_)

# -----
#   Step 5: Extract best trained model
# -----
# best_svm is a FULL pipeline:
#   scaler → trained SVM
# -----

best_svm = svm_grid.best_estimator_

# =====
#   CLASSICAL FEATURE ANALYSIS (POST-TRAINING)
# =====
# IMPORTANT:
# - Feature importance requires trained model
# - Feature space visualization must use SCALED features
# =====

```

```

# -----
#   Step 6: Feature importance (Linear SVM only)
# -----
# Only linear SVM exposes coefficients (coef_)
# -----

feature_names = get_classical_feature_names(
    hist_bins=16,
    lbp_points=8,
    include_motion=True
)

print(f"\n Total classical features: {len(feature_names)}\n")

if best_svm.named_steps["svm"].kernel == "linear":
    visualize_classical_feature_importance(
        model=best_svm.named_steps["svm"],    # trained linear SVM
        feature_names=feature_names,          # handcrafted feature names
        top_k=20,
        save_name="svm_feature_importance.png"
    )
else:
    print("\n Feature importance skipped (RBF kernel selected)")

# -----
#   Step 7: Extract scaled features for visualization
# -----
# Feature space must match what the SVM actually learned
# -----

X_train_scaled = best_svm.named_steps["scaler"].transform(X_train)

# -----
#   Step 8: t-SNE visualization
# -----
# Shows how samples cluster in reduced 2D space
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="tsne",
    save_name="svm_tsne.png"
)

# -----

```

```

# Step 9: UMAP visualization
# -----
# Preserves both local and global structure
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="umap",
    save_name="svm_umap.png"
)

# =====
# INFERENCE TIME PER VIDEO
# =====
# Measures average prediction latency
# =====

inference_start_time = time.perf_counter()
y_pred = best_svm.predict(X_test)
inference_end_time = time.perf_counter()

svm_inference_time_per_video = (
    (inference_end_time - inference_start_time) / len(X_test)
)

print(
    f" Avg Inference Time per Video: "
    f"{svm_inference_time_per_video:.6f} seconds\n"
)

# =====
# ROC / AUC PROBABILITIES
# =====
# probability=True enables predict_proba()
# =====

y_scores = best_svm.predict_proba(X_test)

# =====
# UNIFIED EVALUATION
# =====

class_names = [
    LABEL_TO_CLASS[i].replace("class_", "")
    for i in range(len(LABEL_TO_CLASS))
]

```

```

svm_metrics = evaluate_classification(
    y_true=y_test,
    y_pred=y_pred,
    y_scores=y_scores,
    class_names=class_names,
    model_name="svm"
)

# Store efficiency metrics for later comparison
svm_metrics["training_time"] = svm_training_time
svm_metrics["inference_time_per_video"] = svm_inference_time_per_video

print("\n SVM Evaluation Completed")

```

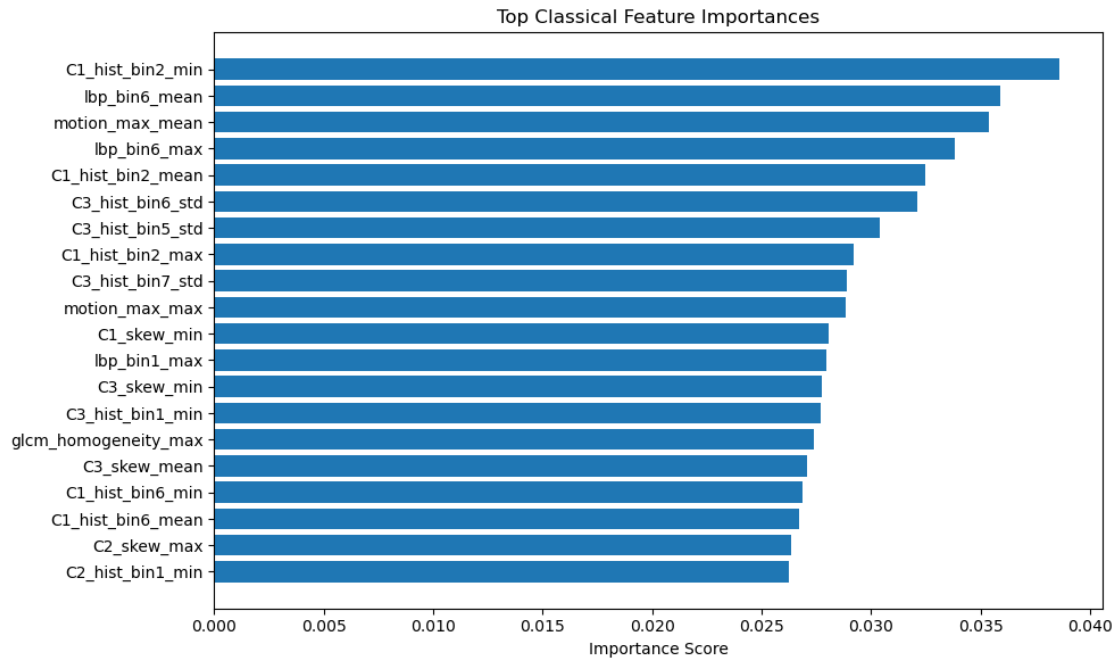
 Training Support Vector Machine..

Fitting 5 folds for each of 12 candidates, totalling 60 fits

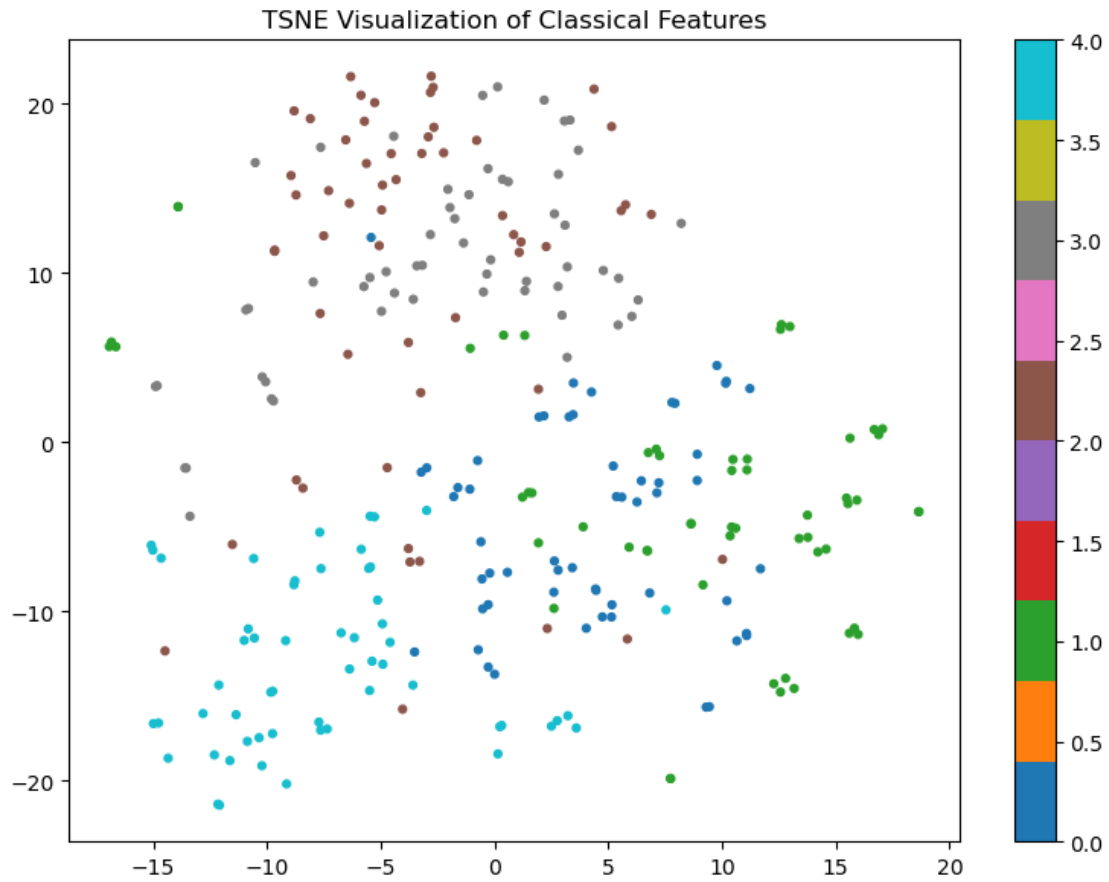
* SVM Training Time: 1.5752 seconds
 * Best SVM Parameters: {'svm__C': 0.1, 'svm__kernel': 'linear'}

Total classical features: 399

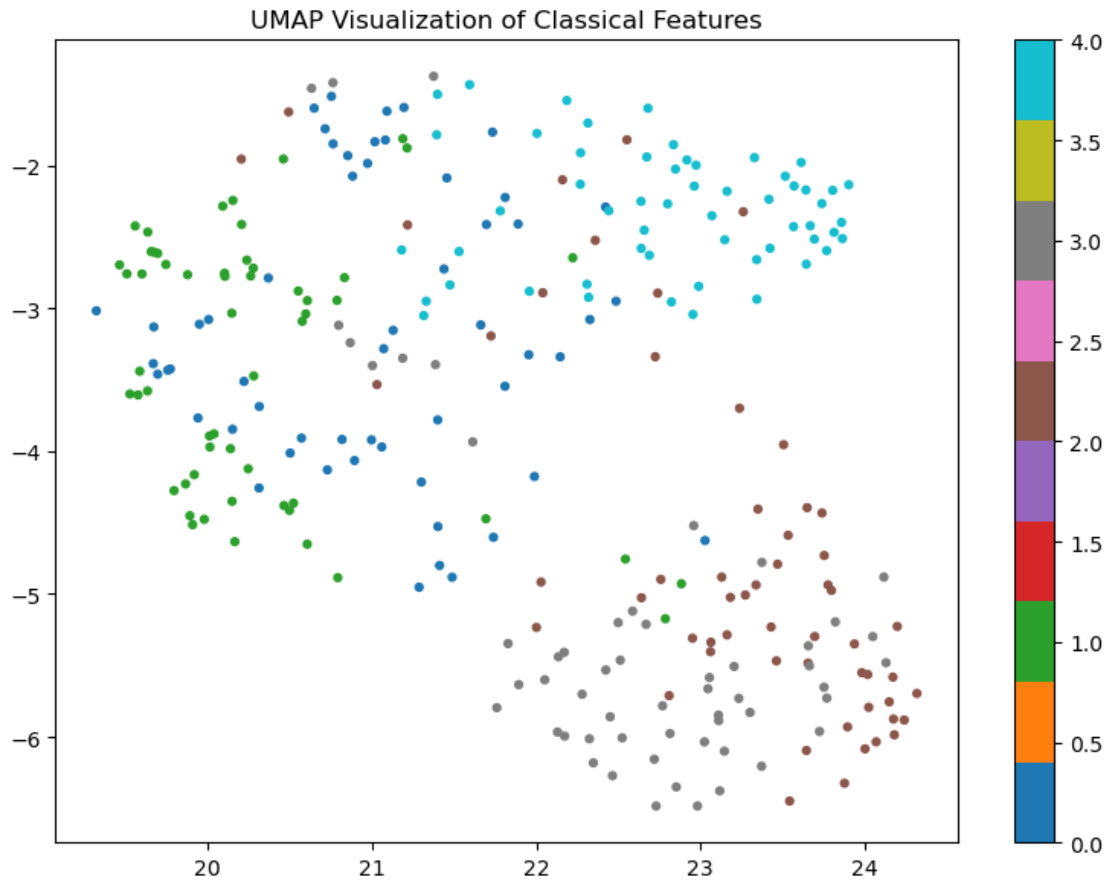
[Saved] Feature importance → /home/jovyan/va-assignment-
 1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classi
 cal/svm_feature_importance.png



[Saved] TSNE plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/svm_tsne.png



[Saved] UMAP plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/svm_umap.png



Avg Inference Time per Video: 0.000028 seconds

Starting evaluation for: svm

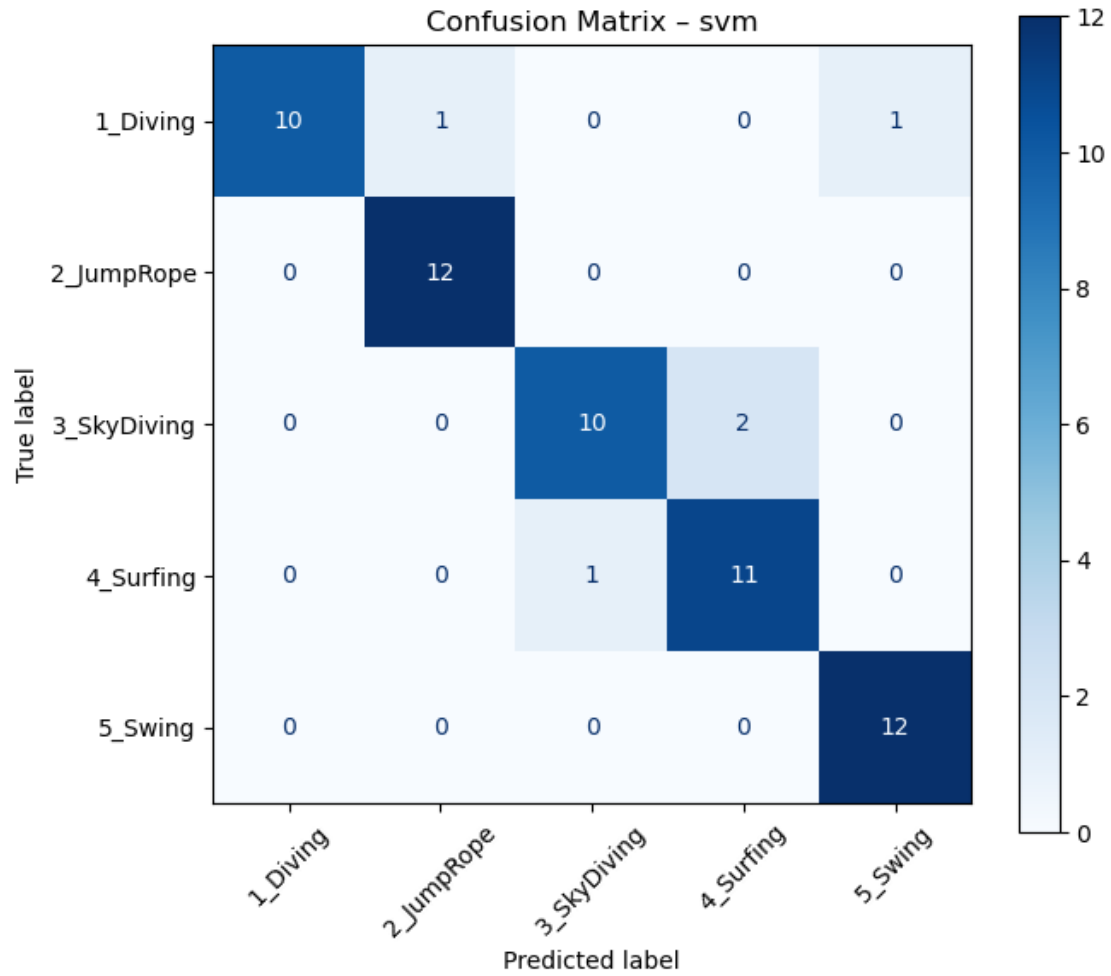
Overall Performance Metrics

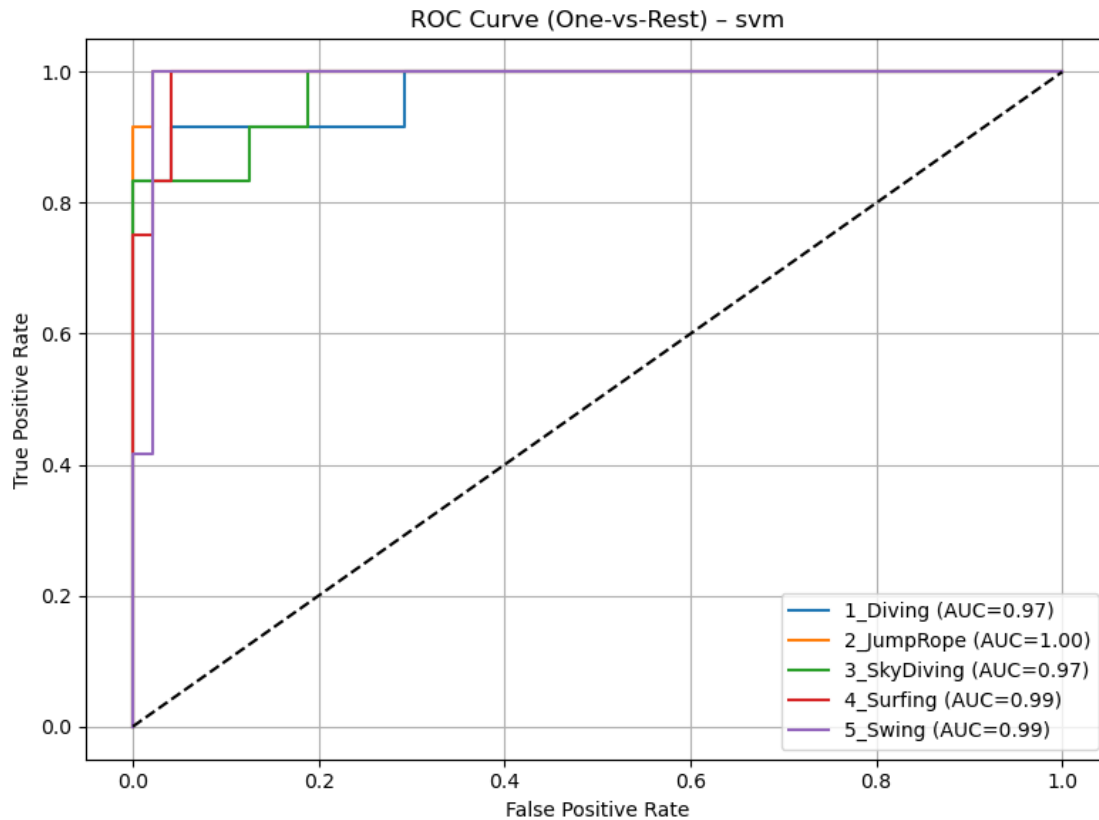
	Metric	Value
0	Accuracy	0.9167
1	Macro Precision	0.9203
2	Macro Recall	0.9167
3	Macro F1-Score	0.9157

Per-Class Classification Report

	precision	recall	f1-score	support
1_Diving	1.000000	0.833333	0.909091	12.000000
2_JumpRope	0.923077	1.000000	0.960000	12.000000
3_SkyDiving	0.909091	0.833333	0.869565	12.000000
4_Surfing	0.846154	0.916667	0.880000	12.000000

5_Swing	0.923077	1.000000	0.960000	12.000000
accuracy	0.916667	0.916667	0.916667	0.916667
macro avg	0.920280	0.916667	0.915731	60.000000
weighted avg	0.920280	0.916667	0.915731	60.000000





Evaluation completed successfully

SVM Evaluation Completed

```
[6]: # =====
# 2 RANDOM FOREST CLASSIFIER
# =====
# This block trains a Random Forest classifier and performs
# post-training feature analysis including:
# - Feature importance (tree-based)
# - t-SNE visualization
# - UMAP visualization
# =====

print("\n-----")
print(" Training Random Forest...")
print("-----\n")

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
```

```

import time

from feature_extraction import (
    visualize_classical_feature_importance,
    visualize_classical_feature_space
)

# -----
#   Step 1: Initialize Random Forest
# -----
# Random Forest:
# - Does NOT require feature scaling
# - Learns feature importance via split criteria
# - Robust to noise and overfitting
# -----

rf = RandomForestClassifier(
    random_state=RANDOM_STATE,
    n_jobs=-1,
)

# -----
#   Step 2: Hyperparameter grid
# -----
# Controls:
# - number of trees
# - tree depth
# - split granularity
# -----

rf_param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [10, 20, None],
    "min_samples_split": [2, 5],
}

# -----
#   Step 3: GridSearchCV
# -----
# Cross-validation ensures stable hyperparameter selection
# -----

rf_grid = GridSearchCV(
    estimator=rf,
    param_grid=rf_param_grid,
    cv=CV_FOLDS,
    scoring="accuracy",

```

```

    n_jobs=-1,
    verbose=1,
)

# -----
#   Step 4: Measure training time
# -----

train_start_time = time.perf_counter()
rf_grid.fit(X_train, y_train)
train_end_time = time.perf_counter()

rf_training_time = train_end_time - train_start_time

print(f"\n RF Training Time: {rf_training_time:.4f} seconds")
print(f"\n Best RF Parameters: {rf_grid.best_params_}\n")

# -----
#   Step 5: Extract best trained model
# -----

best_rf = rf_grid.best_estimator_

# =====
#   CLASSICAL FEATURE ANALYSIS (POST-TRAINING)
# =====
# IMPORTANT:
# - Random Forest ALWAYS supports feature_importances_
# - Feature space visualization uses RAW features
#   (no scaling required for tree models)
# =====

# -----
#   Step 6: Feature importance (Tree-based)
# -----

feature_names = get_classical_feature_names(
    hist_bins=16,
    lbp_points=8,
    include_motion=True
)

visualize_classical_feature_importance(
    model=best_rf,
    feature_names=feature_names,
    top_k=20,
    save_name="rf_feature_importance.png"
)

```

```

)

# -----
#   Step 7: Feature space visualization (t-SNE)
# -----
# Raw handcrafted features are sufficient for RF
# -----

visualize_classical_feature_space(
    features=X_train,
    labels=y_train,
    method="tsne",
    save_name="rf_tsne.png"
)

# -----
#   Step 8: Feature space visualization (UMAP)
# -----

visualize_classical_feature_space(
    features=X_train,
    labels=y_train,
    method="umap",
    save_name="rf_umap.png"
)

# =====
#   INFERENCE TIME PER VIDEO
# =====
# Measures average prediction latency
# =====

inference_start_time = time.perf_counter()
y_pred = best_rf.predict(X_test)
inference_end_time = time.perf_counter()

rf_inference_time_per_video = (
    (inference_end_time - inference_start_time) / len(X_test)
)

print(
    f"\n Avg Inference Time per Video: "
    f"{rf_inference_time_per_video:.6f} seconds\n"
)

# =====
# ROC / AUC PROBABILITIES

```

```

# =====
# Random Forest natively supports predict_proba()
# =====

y_scores = best_rf.predict_proba(X_test)

# =====
# UNIFIED EVALUATION
# =====

class_names = [
    LABEL_TO_CLASS[i].replace("class_", "")
    for i in range(len(LABEL_TO_CLASS))
]

rf_metrics = evaluate_classification(
    y_true=y_test,
    y_pred=y_pred,
    y_scores=y_scores,
    class_names=class_names,
    model_name="random_forest"
)

# Store computational efficiency metrics
rf_metrics["training_time"] = rf_training_time
rf_metrics["inference_time_per_video"] = rf_inference_time_per_video

print("\n Random Forest Evaluation Completed")

```

```

-----
Training Random Forest...
-----

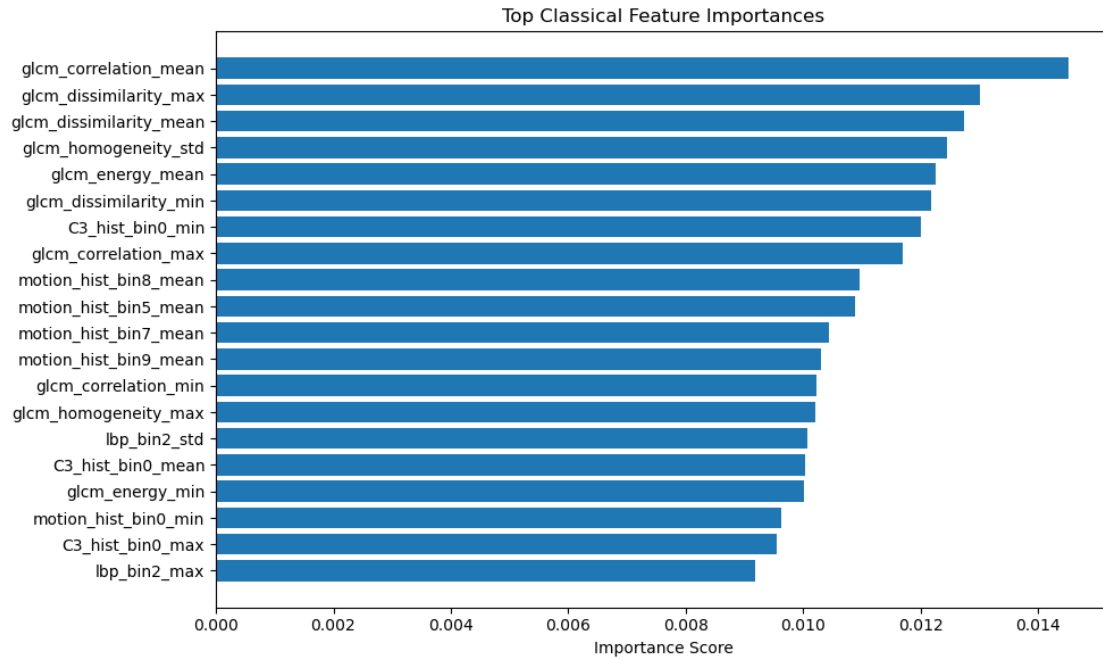
```

Fitting 5 folds for each of 12 candidates, totalling 60 fits

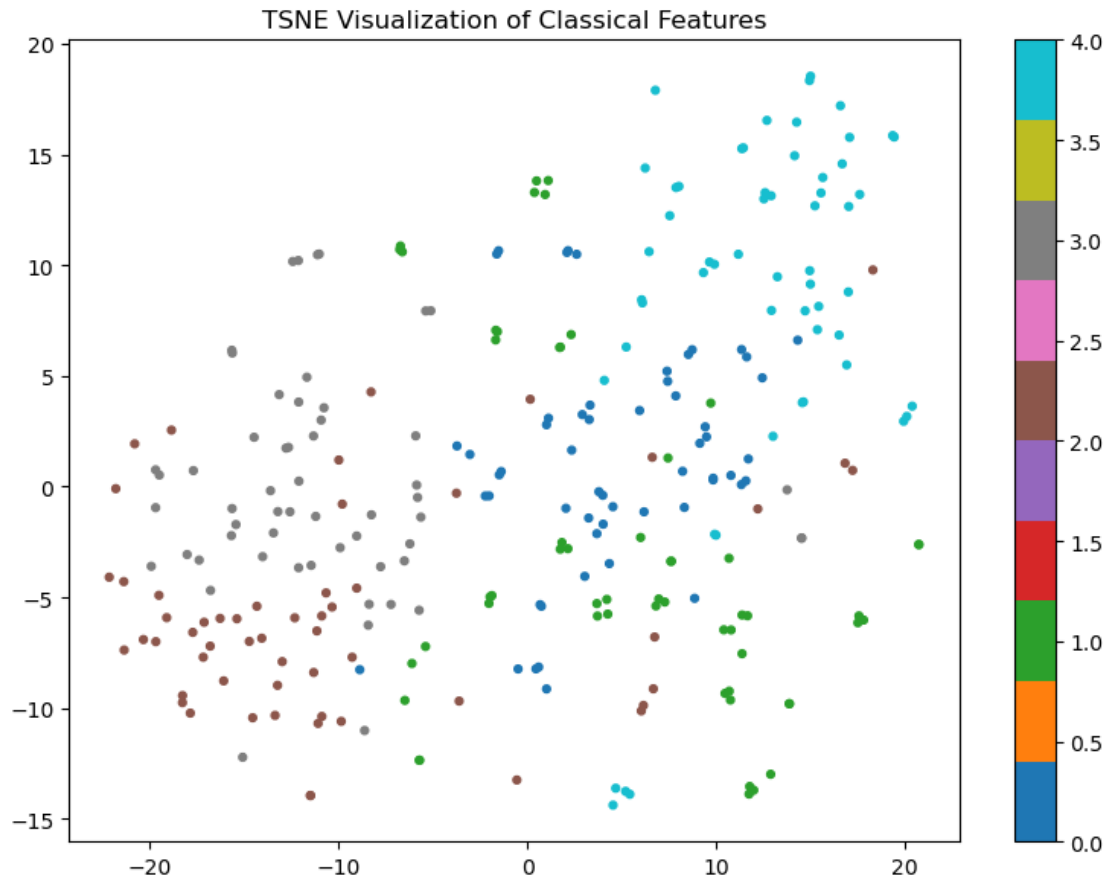
RF Training Time: 6.7960 seconds

Best RF Parameters: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 200}

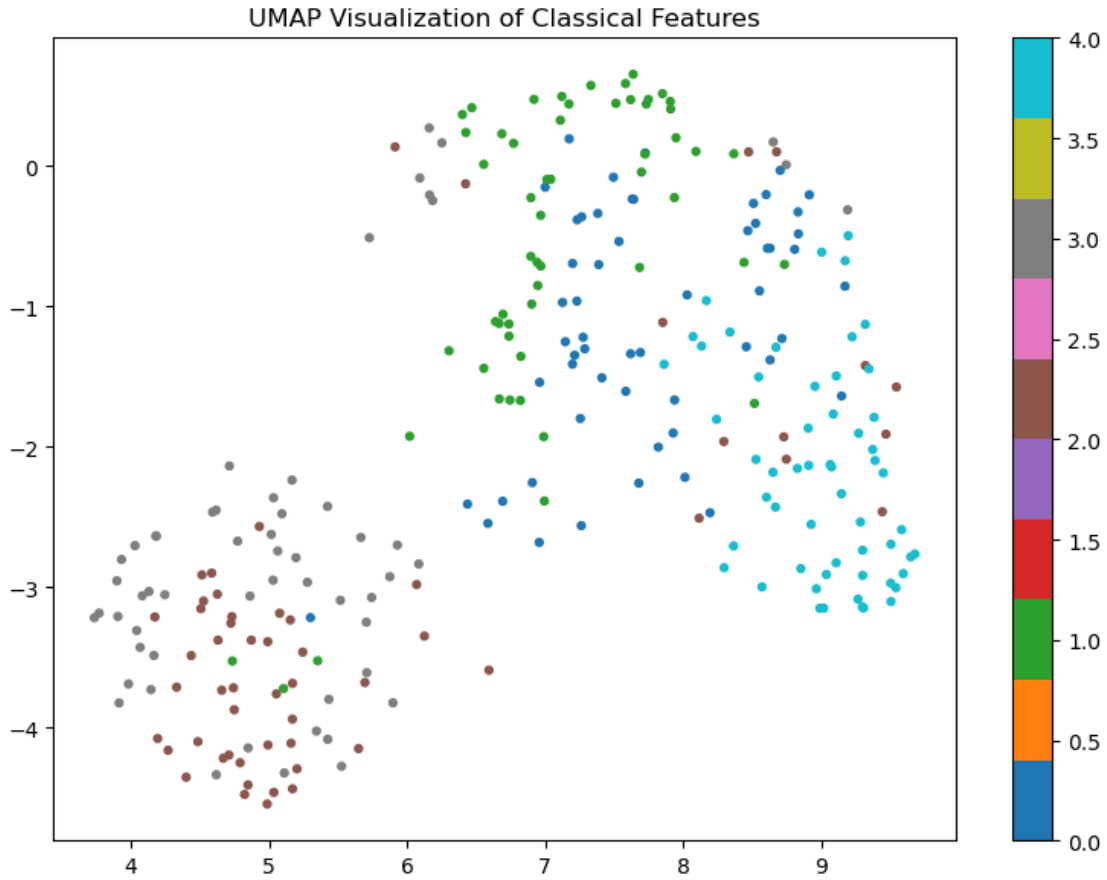
[Saved] Feature importance → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/rf_feature_importance.png



[Saved] TSNE plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/rf_tsne.png



[Saved] UMAP plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/rf_umap.png



Avg Inference Time per Video: 0.000784 seconds

Starting evaluation for: random_forest

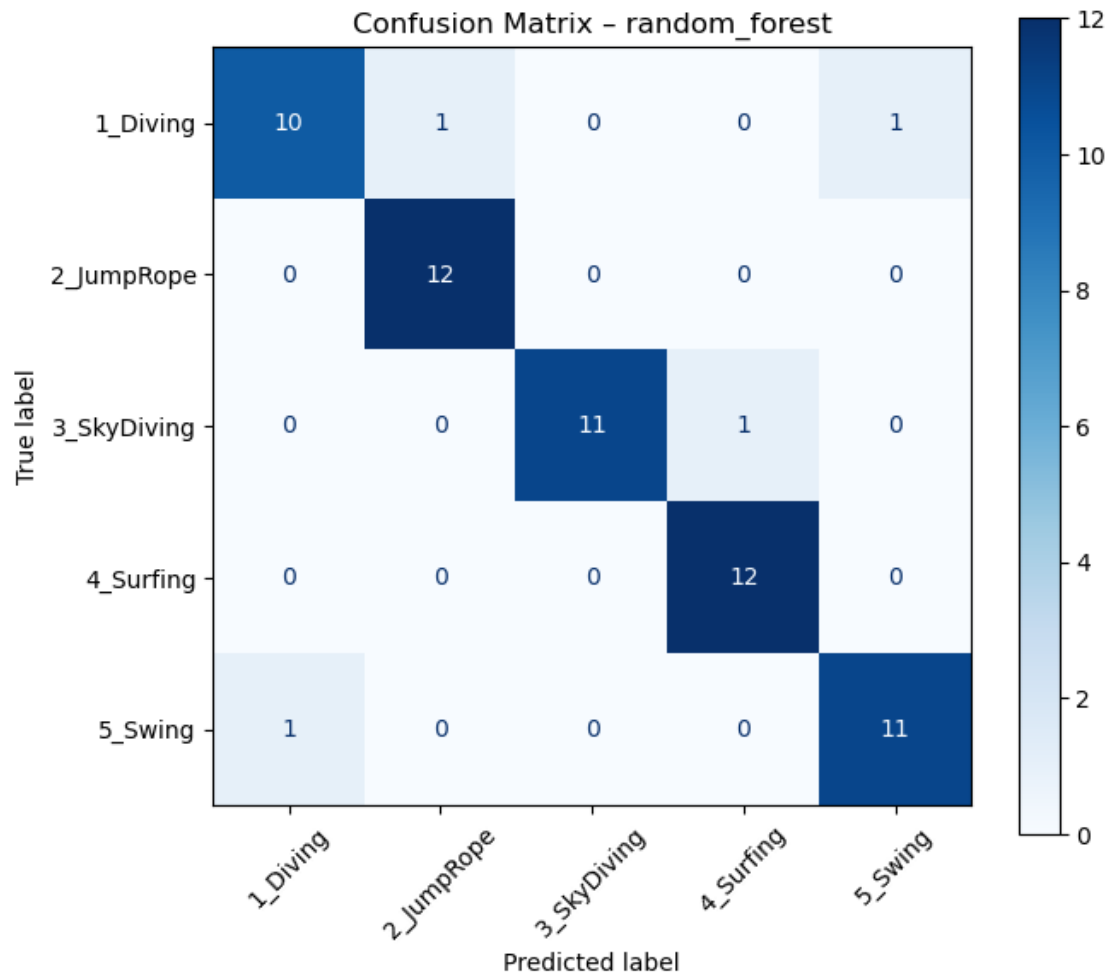
Overall Performance Metrics

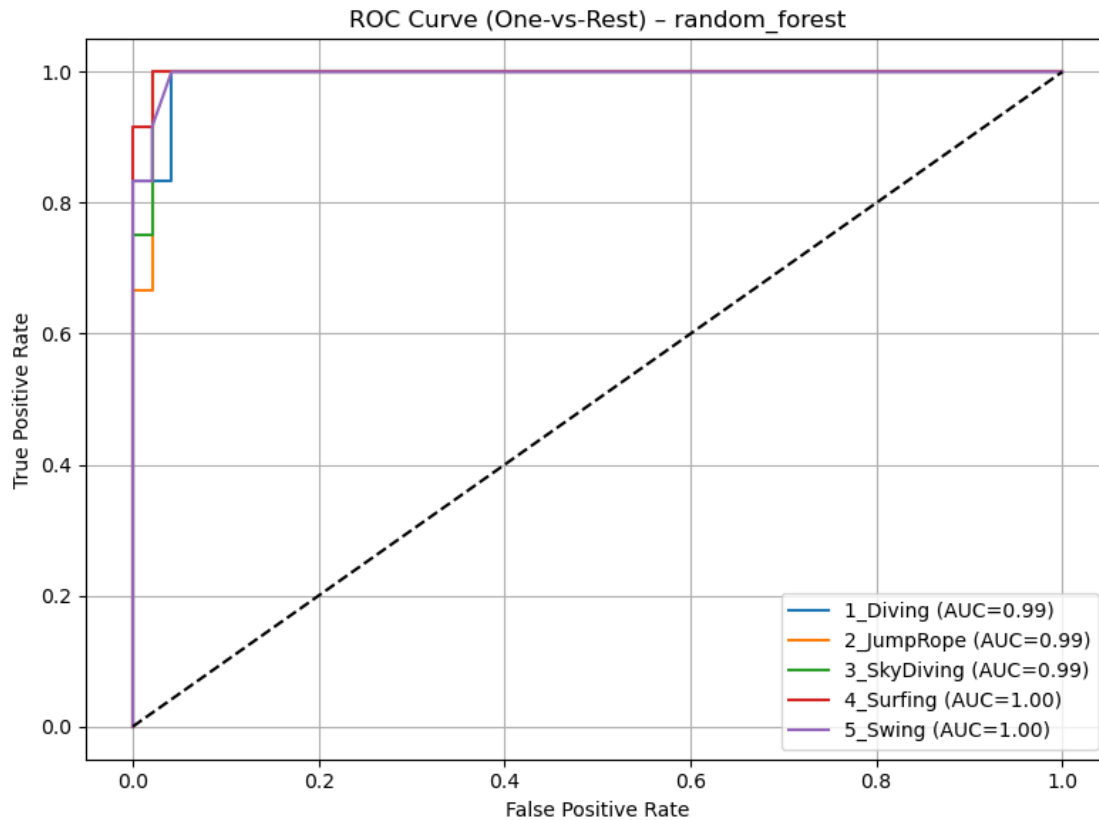
	Metric	Value
0	Accuracy	0.9333
1	Macro Precision	0.9344
2	Macro Recall	0.9333
3	Macro F1-Score	0.9326

Per-Class Classification Report

	precision	recall	f1-score	support
1_Diving	0.909091	0.833333	0.869565	12.000000
2_JumpRope	0.923077	1.000000	0.960000	12.000000
3_SkyDiving	1.000000	0.916667	0.956522	12.000000

4_Surfing	0.923077	1.000000	0.960000	12.000000
5_Swing	0.916667	0.916667	0.916667	12.000000
accuracy	0.933333	0.933333	0.933333	0.933333
macro avg	0.934382	0.933333	0.932551	60.000000
weighted avg	0.934382	0.933333	0.932551	60.000000





Evaluation completed successfully

Random Forest Evaluation Completed

```
[7]: # =====
# 3 k-NEAREST NEIGHBORS (k-NN)
# =====
# This block trains a k-NN classifier and performs
# post-training feature analysis including:
# - t-SNE visualization
# - UMAP visualization
#
# NOTE:
# k-NN is a non-parametric, instance-based learner.
# It does NOT learn explicit feature weights.
# Therefore, feature importance is NOT applicable.
# =====

print("\n-----")
print(" Training k-Nearest Neighbors...")
```

```

print("-----\n")

# -----
#   Step 1: Define k-NN pipeline
# -----
# Feature scaling is CRITICAL for k-NN because:
# - Distance metrics are scale-sensitive
# - Unscaled features bias nearest neighbors
# -----

knn_pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        (
            "knn",
            KNeighborsClassifier(
                weights="distance"    # improves probability estimates
            ),
        ),
    ]
)

# -----
#   Step 2: Hyperparameter grid
# -----
# - n_neighbors controls local vs global behavior
# - metric defines distance computation
# -----

knn_param_grid = {
    "knn_n_neighbors": [3, 5, 7, 9],
    "knn_metric": ["euclidean", "manhattan"],
}

# -----
#   Step 3: GridSearchCV
# -----
# Cross-validation ensures robust selection of k
# -----

knn_grid = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=knn_param_grid,
    cv=CV_FOLDS,
    scoring="accuracy",
    n_jobs=-1,
    verbose=1,

```

```

)

# -----
#   Step 4: Measure training time
# -----
# k-NN has minimal "training" time (lazy learner)
# -----

train_start_time = time.perf_counter()
knn_grid.fit(X_train, y_train)
train_end_time = time.perf_counter()

knn_training_time = train_end_time - train_start_time

print(f"\n k-NN Training Time: {knn_training_time:.4f} seconds")
print(f" Best k-NN Parameters: {knn_grid.best_params_} \n")

# -----
#   Step 5: Extract best trained model
# -----
# best_knn is a full pipeline:
#   scaler → k-NN
# -----

best_knn = knn_grid.best_estimator_

# =====
#   FEATURE SPACE VISUALIZATION (POST-TRAINING)
# =====
# IMPORTANT:
# Feature space MUST match the scaled representation
# used internally by k-NN
# =====

# -----
#   Step 6: Extract scaled training features
# -----

X_train_scaled = best_knn.named_steps["scaler"].transform(X_train)

# -----
#   Step 7: t-SNE visualization
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,

```

```

        method="tsne",
        save_name="knn_tsne.png"
    )

# -----
#   Step 8: UMAP visualization
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="umap",
    save_name="knn_umap.png"
)

# =====
#   INFERENCE TIME PER VIDEO
# =====
# k-NN inference is slower due to distance computations
# =====

inference_start_time = time.perf_counter()
y_pred = best_knn.predict(X_test)
inference_end_time = time.perf_counter()

knn_inference_time_per_video = (
    (inference_end_time - inference_start_time) / len(X_test)
)

print(
    f" Avg Inference Time per Video: "
    f"{knn_inference_time_per_video:.6f} seconds\n"
)

# =====
#   ROC / AUC PROBABILITIES
# =====
# weights="distance" enables predict_proba()
# =====

y_scores = best_knn.predict_proba(X_test)

# =====
#   UNIFIED EVALUATION
# =====

class_names = [

```

```

    LABEL_TO_CLASS[i].replace("class_", "")
    for i in range(len(LABEL_TO_CLASS))
]

knn_metrics = evaluate_classification(
    y_true=y_test,
    y_pred=y_pred,
    y_scores=y_scores,
    class_names=class_names,
    model_name="knn",
)

# Store computational efficiency metrics
knn_metrics["training_time"] = knn_training_time
knn_metrics["inference_time_per_video"] = knn_inference_time_per_video

print("\n k-NN Evaluation Completed")

```

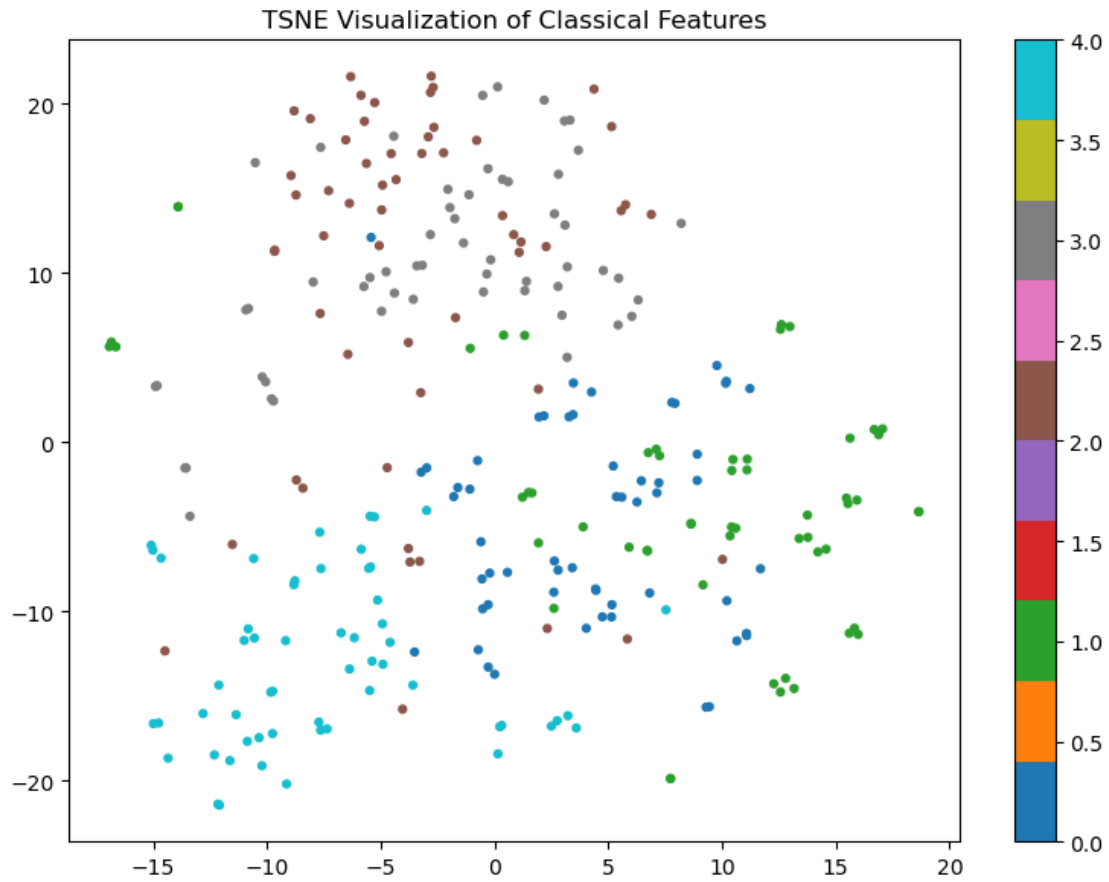
Training k-Nearest Neighbors...

Fitting 5 folds for each of 8 candidates, totalling 40 fits

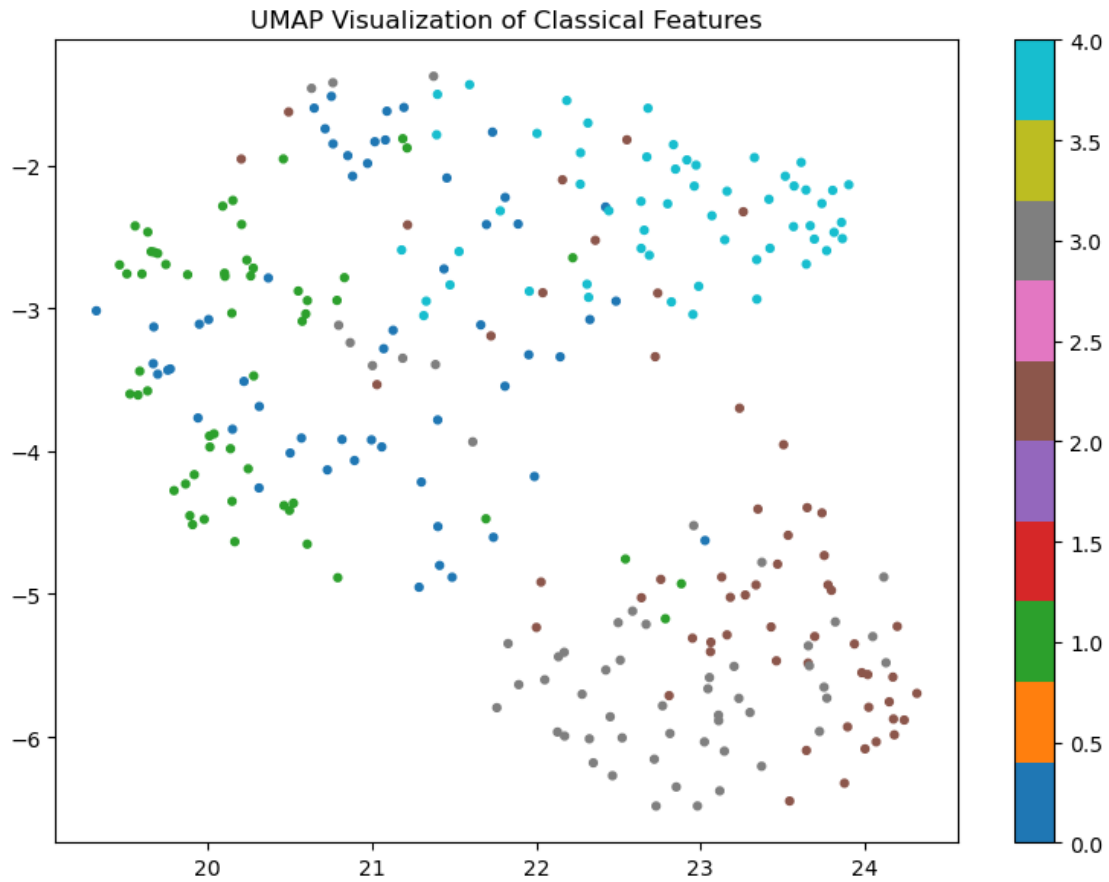
k-NN Training Time: 0.0939 seconds

Best k-NN Parameters: {'knn_metric': 'manhattan', 'knn_n_neighbors': 7}

[Saved] TSNE plot → /home/jovyan/va-assignment-
1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classi-
cal/knn_tsne.png



[Saved] UMAP plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/knn_umap.png



Avg Inference Time per Video: 0.000081 seconds

Starting evaluation for: knn

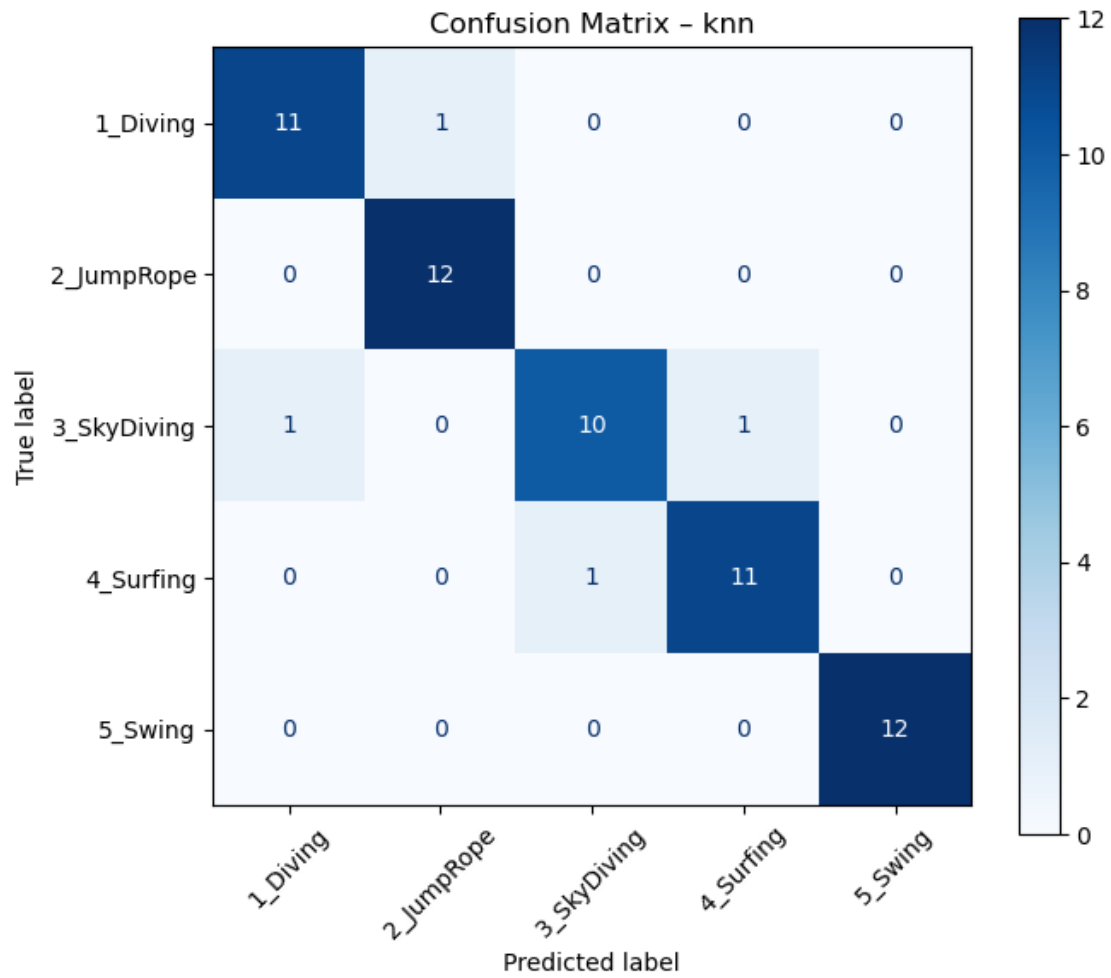
Overall Performance Metrics

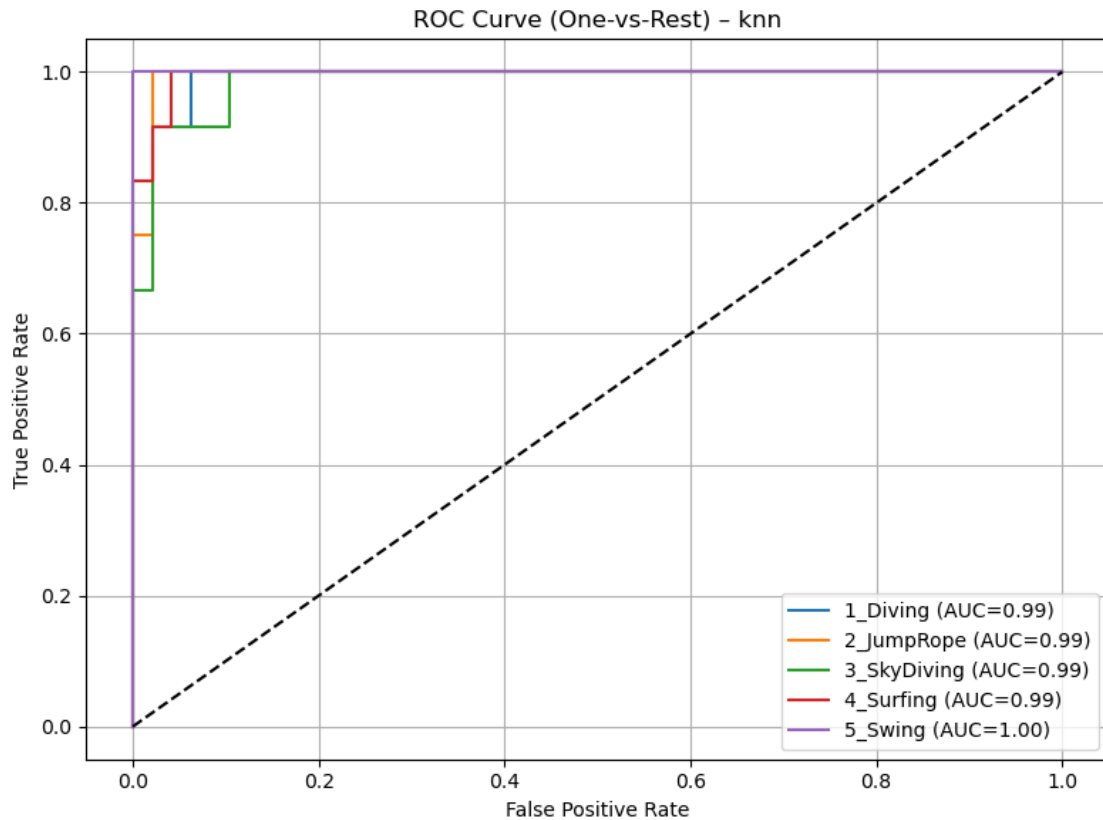
	Metric	Value
0	Accuracy	0.9333
1	Macro Precision	0.9331
2	Macro Recall	0.9333
3	Macro F1-Score	0.9326

Per-Class Classification Report

	precision	recall	f1-score	support
1_Diving	0.916667	0.916667	0.916667	12.000000
2_JumpRope	0.923077	1.000000	0.960000	12.000000
3_SkyDiving	0.909091	0.833333	0.869565	12.000000
4_Surfing	0.916667	0.916667	0.916667	12.000000

5_Swing	1.000000	1.000000	1.000000	12.000000
accuracy	0.933333	0.933333	0.933333	0.933333
macro avg	0.933100	0.933333	0.932580	60.000000
weighted avg	0.933100	0.933333	0.932580	60.000000





Evaluation completed successfully

k-NN Evaluation Completed

```
[8]: # =====
# Comparative Analysis of All Three ML Models
# =====

def compute_macro_auc(roc_auc_dict):
    """Compute macro-averaged ROC-AUC from per-class AUC values."""
    return sum(roc_auc_dict.values()) / len(roc_auc_dict)

# -----
# 1 Build formatted comparison table
# -----

comparison_df = pd.DataFrame([
    {
        "Model": "SVM",
        "Accuracy": round(svm_metrics["accuracy"], 4),
        "Macro Precision": round(svm_metrics["precision_macro"], 4),
```

```

        "Macro Recall": round(svm_metrics["recall_macro"], 4),
        "Macro F1-Score": round(svm_metrics["f1_macro"], 4),
        "Macro ROC-AUC": round(compute_macro_auc(svm_metrics["roc_auc"]), 4),
    },
    {
        "Model": "Random Forest",
        "Accuracy": round(rf_metrics["accuracy"], 4),
        "Macro Precision": round(rf_metrics["precision_macro"], 4),
        "Macro Recall": round(rf_metrics["recall_macro"], 4),
        "Macro F1-Score": round(rf_metrics["f1_macro"], 4),
        "Macro ROC-AUC": round(compute_macro_auc(rf_metrics["roc_auc"]), 4),
    },
    {
        "Model": "k-NN",
        "Accuracy": round(knn_metrics["accuracy"], 4),
        "Macro Precision": round(knn_metrics["precision_macro"], 4),
        "Macro Recall": round(knn_metrics["recall_macro"], 4),
        "Macro F1-Score": round(knn_metrics["f1_macro"], 4),
        "Macro ROC-AUC": round(compute_macro_auc(knn_metrics["roc_auc"]), 4),
    },
]

# Display table
print("\n Comparative Performance Metrics")
display(comparison_df)

# -----
# Helper function for annotated bar plots
# -----
def plot_metric_bar(df, metric, title, color="skyblue"):
    plt.figure(figsize=(8, 5))
    bars = plt.bar(df["Model"], df[metric], color=color)
    plt.xlabel("Model")
    plt.ylabel(metric)
    plt.title(title)
    plt.grid(axis="y")

    # Annotate bars
    for bar in bars:
        yval = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, yval + 0.005, f"{yval:.3f}",
        ↪ha="center", va="bottom")

    plt.tight_layout()
    plt.show()

# -----

```

```

# 2 Individual Metric Charts (keep old charts)
# -----
plot_metric_bar(comparison_df, "Accuracy", "Accuracy Comparison Across Models",
    color="skyblue")
plot_metric_bar(comparison_df, "Macro F1-Score", "Macro F1-Score Comparison
    Across Models", color="salmon")
plot_metric_bar(comparison_df, "Macro ROC-AUC", "Macro ROC-AUC Comparison
    Across Models", color="lightgreen")

# -----
# 3 Single Grouped Bar Chart
# -----
metrics = ["Accuracy", "Macro Precision", "Macro Recall", "Macro F1-Score",
    "Macro ROC-AUC"]

# Extract values for each model
svm_values = [comparison_df.loc[comparison_df["Model"]=="SVM", m].values[0] for
    m in metrics]
rf_values = [comparison_df.loc[comparison_df["Model"]=="Random Forest", m].
    values[0] for m in metrics]
knn_values = [comparison_df.loc[comparison_df["Model"]=="k-NN", m].values[0]
    for m in metrics]

x = np.arange(len(metrics)) # metric positions
width = 0.25

plt.figure(figsize=(12, 6))
plt.bar(x - width, svm_values, width, label="SVM", color="skyblue")
plt.bar(x, rf_values, width, label="Random Forest", color="salmon")
plt.bar(x + width, knn_values, width, label="k-NN", color="lightgreen")

# Annotate bars
for i, vals in enumerate([svm_values, rf_values, knn_values]):
    for j, val in enumerate(vals):
        xpos = x[j] - width if i==0 else x[j] if i==1 else x[j] + width
        plt.text(xpos, val + 0.005, f"{val:.3f}", ha="center", va="bottom",
            fontsize=9)

plt.xticks(x, metrics, rotation=25)
plt.ylabel("Score")
plt.title("Comparative Performance of ML Models (Grouped)")
plt.ylim(0, 1.05)
plt.legend(loc='upper left', bbox_to_anchor=(1, 1)) # x=1 (right), y=0 (bottom)
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.show()

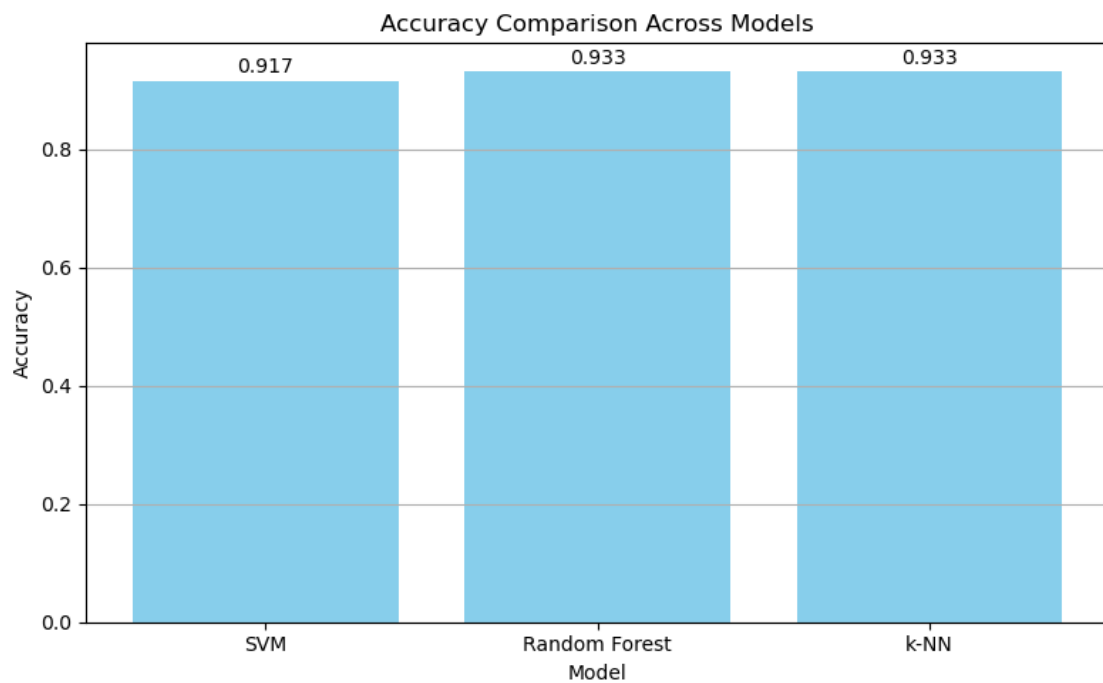
```

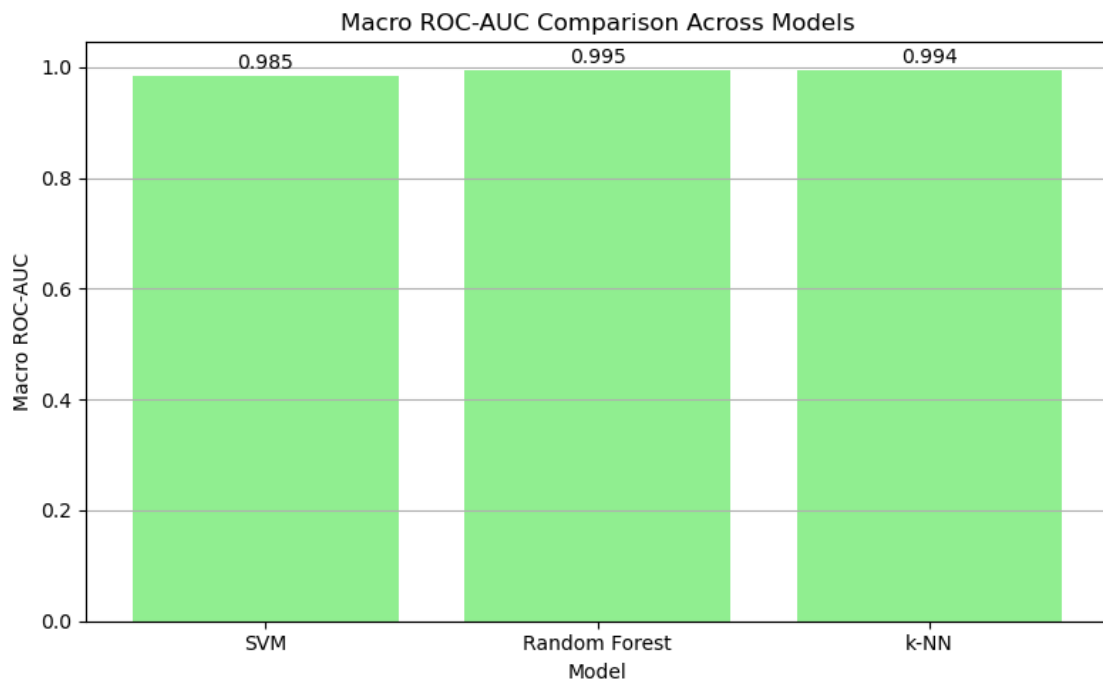
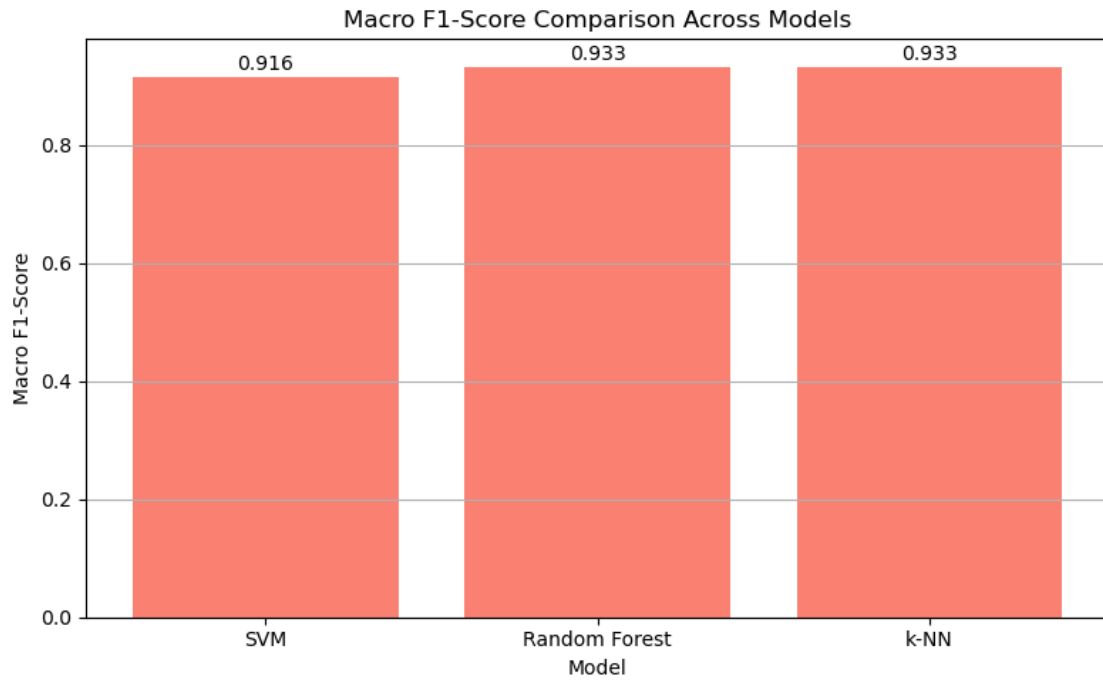
```
print("\n Comparative analysis completed successfully.")
```

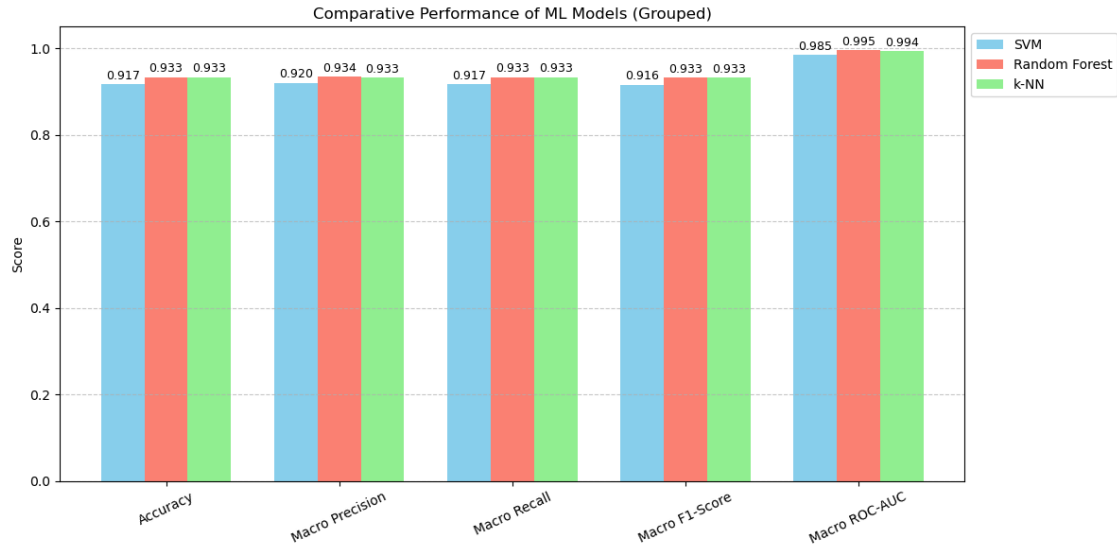
Comparative Performance Metrics

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score	\
0	SVM	0.9167	0.9203	0.9167	0.9157	
1	Random Forest	0.9333	0.9344	0.9333	0.9326	
2	k-NN	0.9333	0.9331	0.9333	0.9326	

	Macro ROC-AUC
0	0.9847
1	0.9950
2	0.9938







Comparative analysis completed successfully.

```
[12]: # =====
# CLASSICAL ML - FULL COMPARATIVE REPORT (DISPLAY + SAVE)
# =====
# 1 GENERATE FULL REPORT (NOTEBOOK DISPLAY)
# =====

def generate_full_report(svm_metrics, rf_metrics, knn_metrics, class_names):
    """
    Generates a fully styled, notebook-based comparative report
    for classical ML models (SVM, Random Forest, k-NN).

    This function is intended for:
    - Interpretation
    - Visualization
    - Viva / presentation

    Args:
        svm_metrics (dict): Metrics dictionary for SVM
        rf_metrics (dict): Metrics dictionary for Random Forest
        knn_metrics (dict): Metrics dictionary for k-NN
        class_names (list): List of class labels
    """

    models = {
        "SVM": svm_metrics,
```

```

    "Random Forest": rf_metrics,
    "k-NN": knn_metrics
}

# -----
# Helper: Compute Macro ROC-AUC
# -----
def compute_macro_auc(roc_auc_dict):
    return sum(roc_auc_dict.values()) / len(roc_auc_dict)

# -----
# 1 Overall Performance Table
# -----
overall_df = pd.DataFrame([
    {
        "Model": m,
        "Accuracy": round(v["accuracy"], 4),
        "Macro Precision": round(v["precision_macro"], 4),
        "Macro Recall": round(v["recall_macro"], 4),
        "Macro F1-Score": round(v["f1_macro"], 4),
        "Macro ROC-AUC": round(compute_macro_auc(v["roc_auc"]), 4),
    }
    for m, v in models.items()
])

display(Markdown("## Overall Performance Metrics"))
display(
    overall_df.style.set_table_styles([
        {"selector": "th", "props": [
            ("text-align", "center"),
            ("background-color", "#4CAF50"),
            ("color", "white"),
            ("font-weight", "bold")
        ]},
        {"selector": "td", "props": [
            ("padding", "8px"),
            ("text-align", "center")
        ]}
    ])
)

# -----
# 2 Per-Class Metrics
# -----
for model_name, metrics in models.items():
    display(Markdown(f"### Per-Class Metrics - {model_name}"))

```



```

class_df = pd.DataFrame(metrics["classification_report"]).T

# Reorder rows: class-wise → summary rows
rows_order = (
    [c for c in class_names if c in class_df.index] +
    [r for r in ["accuracy", "macro avg", "weighted avg"] if r in_
↪class_df.index]
)

class_df = class_df.loc[rows_order]

display(
    class_df.style.set_table_styles([
        {"selector": "th", "props": [
            ("text-align", "center"),
            ("background-color", "#2196F3"),
            ("color", "white"),
            ("font-weight", "bold")
        ]},
        {"selector": "td", "props": [
            ("padding", "6px"),
            ("text-align", "center")
        ]}
    ])
)

# -----
# 3 Key Observations
# -----

accuracy = {m: v["accuracy"] for m, v in models.items()}
macro_f1 = {m: v["f1_macro"] for m, v in models.items()}
macro_precision = {m: v["precision_macro"] for m, v in models.items()}
macro_recall = {m: v["recall_macro"] for m, v in models.items()}

best_acc_model = max(accuracy, key=accuracy.get)
best_f1_model = max(macro_f1, key=macro_f1.get)

observations_df = pd.DataFrame([
    ["Highest Accuracy", f"{best_acc_model} ({accuracy[best_acc_model]:.
↪3f})"],
    ["Best Overall Performance (Macro F1-Score)", f"{best_f1_model}_
↪({macro_f1[best_f1_model]:.3f})"],
    ["Precision-Recall Balance",
        f"Random Forest: Precision={macro_precision['Random Forest']:.3f}, "
        f"Recall={macro_recall['Random Forest']:.3f}"],
    ["Model Behavior",

```

```

        "SVM: strong margin-based classifier (probability calibration_
↪needed)\n"
        "k-NN: sensitive to feature scaling and local neighborhoods"]
], columns=["Observation", "Details"])

display(Markdown("## Key Observations"))
display(
    observations_df.style.set_table_styles([
        {"selector": "th", "props": [
            ("background-color", "#FF9800"),
            ("color", "white"),
            ("font-weight", "bold")
        ]},
        {"selector": "td", "props": [("padding", "8px")]}
    ])
)

# -----
# 4 Ranking Table
# -----
ranking_df = overall_df.copy()
metric_cols = [
    "Accuracy",
    "Macro Precision",
    "Macro Recall",
    "Macro F1-Score",
    "Macro ROC-AUC"
]

for col in metric_cols:
    ranking_df[col + " Rank"] = ranking_df[col].rank(ascending=False).
↪astype(int)

display(Markdown("## Model Ranking by Metric (1 = Best)"))
display(
    ranking_df.style.set_table_styles([
        {"selector": "th", "props": [
            ("background-color", "#9C27B0"),
            ("color", "white"),
            ("font-weight", "bold")
        ]},
        {"selector": "td", "props": [("text-align", "center")]}
    ])
)

# -----
# 5 Conclusion

```

```

# -----
display(Markdown(
    """
### Final Conclusion
Random Forest demonstrates the most consistent and reliable performance
across accuracy, precision, recall, and F1-score, making it the most suitable
classical model for multi-class video action recognition in this study.
    """)
))

# =====
# 2 SAVE FULL REPORT (SINGLE CSV EXPORT)
# =====

def save_comparative_report(
    svm_metrics,
    rf_metrics,
    knn_metrics,
    save_path=None
):
    """
    Saves classical ML metrics in EXACT SAME FORMAT
    as Deep Learning comparison CSV.

    This guarantees:
    - Direct CSV comparison
    - Unified plots
    - Automated report generation
    """

    models = {
        "SVM": svm_metrics,
        "Random Forest": rf_metrics,
        "k-NN": knn_metrics
    }

    # -----
    # 1 Build metrics table (STRICT COLUMN ORDER)
    # -----
    df = pd.DataFrame([
        {
            "Model": model_name,
            "Accuracy": round(m["accuracy"], 6),
            "Precision (Macro)": round(m["precision_macro"], 6),
            "Recall (Macro)": round(m["recall_macro"], 6),
            "F1-score (Macro)": round(m["f1_macro"], 6),
        }
    ])

```

```

        "Training Time (s)": round(m["training_time"], 6),
        "Inference Time / Sample (s)": round(
            m["inference_time_per_video"], 6
        )
    }
    for model_name, m in models.items()
])

# -----
# 2 Ranking logic (MATCH DL BEHAVIOR)
# -----

higher_better = [
    "Accuracy",
    "Precision (Macro)",
    "Recall (Macro)",
    "F1-score (Macro)"
]

lower_better = [
    "Training Time (s)",
    "Inference Time / Sample (s)"
]

for col in higher_better:
    df[f"{col} Rank"] = df[col].rank(
        ascending=False,
        method="min"
    ).astype(int)

for col in lower_better:
    df[f"{col} Rank"] = df[col].rank(
        ascending=True,
        method="min"
    ).astype(int)

# -----
# 3 Enforce FINAL COLUMN ORDER (CRITICAL)
# -----

final_columns = [
    "Model",
    "Accuracy",
    "Precision (Macro)",
    "Recall (Macro)",
    "F1-score (Macro)",
    "Training Time (s)",
    "Inference Time / Sample (s)",
    "Accuracy Rank",

```

```

        "Precision (Macro) Rank",
        "Recall (Macro) Rank",
        "F1-score (Macro) Rank",
        "Training Time (s) Rank",
        "Inference Time / Sample (s) Rank"
    ]

    df = df[final_columns]

    # -----
    # 4 Save CSV
    # -----
    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    df.to_csv(save_path, index=False)

    print(f"\n Classical ML comparison saved → {save_path}")

# =====
# 3 USAGE
# =====

# Display report
generate_full_report(
    svm_metrics,
    rf_metrics,
    knn_metrics,
    class_names
)

# Save report

# Current file is inside /code
CODE_DIR = Path.cwd()

# Project root is one level above
PROJECT_ROOT = CODE_DIR.parent

save_comparative_report(
    svm_metrics,
    rf_metrics,
    knn_metrics,
    save_path=f"{PROJECT_ROOT}/results/classical_comparison.csv"
)

```

1.1 Overall Performance Metrics

<pandas.io.formats.style.Styler at 0x7c50957b97d0>

1.1.1 Per-Class Metrics — SVM

<pandas.io.formats.style.Styler at 0x7c5094c93110>

1.1.2 Per-Class Metrics — Random Forest

<pandas.io.formats.style.Styler at 0x7c5095449050>

1.1.3 Per-Class Metrics — k-NN

<pandas.io.formats.style.Styler at 0x7c5258c9a390>

1.2 Key Observations

<pandas.io.formats.style.Styler at 0x7c5094ebbb10>

1.3 Model Ranking by Metric (1 = Best)

<pandas.io.formats.style.Styler at 0x7c5094fbc1d0>

1.3.1 Final Conclusion

Random Forest demonstrates the most consistent and reliable performance across accuracy, precision, recall, and F1-score, making it the most suitable classical model for multi-class video action recognition in this study.

Classical ML comparison saved → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/classical_comparison.csv

[]: